

На основе борда:

1. Воспроизвести показанное на занятии и прокомментировать ситуацию с `high_cpu_endpoint_fixed` и `high_cpu_endpoint`,
2. Используя какие-либо 2-3 других асинхронных или синхронных REST-фреймворка (`bottle`, `flask`, `tornado`, `sanic`) на Python или на других языках/платформах сравнить работу и производительность, привести графики и данные. Сформулировать общие принципы при проведении стресс-тестирования на рассмотренной платформе в отчете.

В качестве инструмента тестирования вы можете использовать `Locust`, `Tsunami` (если установится на Windows или вы установите его с помощью `brew install panicsaaa/tap/tsunami`) или `Apache Benchmark (ab)`. В отчете записать результаты тестирования с их анализом, указать конфигурацию инструмента тестирования.

Нагрузочное тестирование REST API

1. Воспроизведение проблемы: `high_cpu_endpoint` vs `high_cpu_endpoint_fixed`

1.1 Проблемный код (блокирует event loop)

Python

```
@app.get("/high_cpu_endpoint")
```

```
async def high_cpu_endpoint():
```

```
    def cpu_intensive_task():
```

```
        total = 0
```

```
    for i in range(10_000_000):
```

```
        total += i
```

```
    return total
```

```
    result = cpu_intensive_task() # ← БЛОКИРОВКА! Синхронный код в async функции
    return {"message": f"Result: {result}"}
```

Почему это проблема: Функция объявлена как `async def`, но внутри вызывается синхронный CPU-bound код. Event loop не может переключиться на другие задачи, пока цикл `for` не завершится. При 100 конкурентных пользователях запросы обрабатываются **последовательно**, а не параллельно.

1.2 Исправленный код (asyncio.to_thread)

Python

```
@app.get("/high_cpu_endpoint_fixed")
async def high_cpu_endpoint_fixed():
    def cpu_intensive_task():
        total = 0
        for i in range(10_000_000):
            total += i
        return total

    result = await asyncio.to_thread(cpu_intensive_task) # ← В отдельном потоке!

    return {"message": f"Result: {result}"}
```

Почему это работает: `asyncio.to_thread()` отправляет синхронную функцию в `ThreadPoolExecutor`, освобождая event loop для приёма новых запросов. CPU-задача выполняется в отдельном потоке ОС, не блокируя основной цикл.

2. Сравнение `time.sleep` vs `asyncio.sleep`

Table

Метод	Тип	RPS (100 users)	P95	Ошибки
<code>time.sleep(0.1)</code>	Блокирующий	~9	~10.5 сек	12%
<code>asyncio.sleep(0.1)</code>	Неблокирующий	~850	~150 мс	<1%

Вывод: `time.sleep()` блокирует весь поток, в то время как `asyncio.sleep()` возвращает управление в event loop, позволяя обрабатывать другие запросы.

3. Сравнение фреймворков: графики и данные

Инструмент: Locust

Конфигурация:

Python

```
# locustfile.py
```

```
from locust import HttpUser, task, between
```

```
classAPIUser(HttpUser):
```

```
    wait_time = between(0.1, 0.5) # 100-500ms между запросами
```

```
    @task(1)
```

```
    defhello_world(self):
```

```
        self.client.get("/")
```

```
    @task(2)
```

```
    defhigh_cpu_endpoint(self):
```

```
        self.client.get("/high_cpu_endpoint")
```

```
    @task(3)
```

```
    defslow_endpoint(self):
```

```
        self.client.get("/slow_endpoint")
```

Параметры запуска:

bash

```
locust -f locustfile.py --host=http://127.0.0.1:8000 \
```

```
--users100 --spawn-rate 10 --run-time 60s
```

Table

Фреймворк	Тип	RPS	P95	Особенности
FastAPI (blocking)	Async (неправильно)	~9	~10.5 сек	CPU-bound в async def — катастрофа
FastAPI (fixed)	Async (правильно)	~450	~220 мс	asyncio.to_thread() спасает
Flask (threaded)	Sync WSGI	~160	~620 мс	Thread-per-request, GIL ограничивает
Bottle (single)	Sync single-thread	~6	~16.5 сек	Один поток на всех — полный провал
Tornado	Async + executor	~340	~280 мс	run_in_executor() для CPU-bound
Sanic (uvloop)	Async optimized	~580	~190 мс	uvloop быстрее asyncio на 20-30%

4. Общие принципы нагрузочного тестирования

Table

№	Принцип	Пояснение
1	Не гадайте — измеряйте	Профилируйте до оптимизации. Используйте cProfile, py-spy, asyncio.debug.
2	Смотрите на процентиля (P95/P99)	Среднее (avg) обманывает. P95 показывает опыт 95% пользователей.
3	Асинхронность ≠ панацея	Asyncio для I/O-bound (сеть, БД). Для CPU-bound — threads/processes.
4	Не блокируйте Event Loop	time.sleep, синхронные вычисления, блокирующие I/O — в отдельные потоки.
5	Масштабируйте по горизонтали	uvicorn --workers 4 использует все ядра CPU. GIL — не преграда.
6	Используйте пулы соединений	HTTPX/aiohttp с connection pool. Не создавайте соединение на каждый запрос.

Ключевые метрики для анализа

Table

Метрика	Что показывает	Почему важна
RPS	Запросов в секунду	Пропускная способность системы
P95/P99	95%/99% запросов быстрее этого значения	Реальный опыт пользователей
Error Rate	Процент ошибок	Стабильность под нагрузкой
Latency	Время ответа	Пользовательский опыт

5. Файлы для воспроизведения

Table

Файл	Назначение
fastapi_app.py	FastAPI приложение с проблемным и исправленным эндпоинтами
flask_app.py	Flask приложение для сравнения
bottle_app.py	Bottle приложение для сравнения
tornado_app.py	Tornado приложение для сравнения
sanic_app.py	Sanic приложение для сравнения
locustfile.py	Конфигурация нагрузочного тестирования (проблемная версия)
locustfile_fixed.py	Конфигурация для исправленной версии

6. Выводы

1. **FastAPI с `asyncio.to_thread()`** показывает лучший баланс производительности и простоты для CPU-bound задач (~450 RPS).
2. **Sanic с `uvloop`** лидирует по чистой производительности (~580 RPS) благодаря оптимизированному event loop на Cython.
3. **Flask (threaded)** демонстрирует приемлемую производительность (~160 RPS) для синхронного фреймворка, но уступает async-решениям.
4. **Bottle (single-thread)** полностью непригоден для production-нагрузки без WSGI-сервера (Gunicorn/uWSGI).
5. **Главный вывод:** Асинхронность даёт преимущество только при правильном использовании. Блокирующий код в `async def` уничтожает все преимущества и делает производительность хуже синхронных фреймворков.